

Exercise 5

Last update February 17, 2026

This exercise sheet must be handed in via LearnIt.

You solve the assignments in your groups.

Your name must be part of the filename, e.g., FP-05-<name>-<name>.fsx.

An example: FP-05-MadsAndersen-HugoHansen.fsx.

You can only upload one file and it must be of type fs or fsx.

It is important that you annotate your own code with comments. It is also important that you apply a functional style, i.e., no loops and no mutable variables.

For this hand-in you also need to consider scenarios where your solutions should return an error, i.e., an exception. The requirement is, that no matter what input you pass to your function that fulfils the function type, then the function should return the intended answer or an exception. It is up to you to define the exceptions and whether they should carry extra information, like error messages.

The exercises 5.1 to 5.3 refer to code defined in the slide deck from lecture 5 about finite trees. You need to look there for the definition of 'a BinTree and in-order traversal.

Exercise 5.1 Consider the definition of type 'a BinTree on slide 30. Write a function

```
inOrder : 'a BinTree -> 'a list
```

that makes an in-order traversal of the tree and collect the elements in a result list. In-order traversal is defined on slide 32.

With the value intBinTree defined on slide 30

```
let intBinTree =
  Node(43, Node(25, Node(56, Leaf, Leaf), Leaf),
        Node(562, Leaf, Node(78, Leaf, Leaf)))
```

we get the following:

```
> inOrder intBinTree;;
val it : int list = [56; 25; 43; 562; 78]
```

Exercise 5.2 Write a function

```
mapInOrder : ('a -> 'b) -> 'a BinTree -> 'b BinTree
```

that makes an in-order traversal of the binary tree and apply the function on all nodes in the tree.

With the value intBinTree

```
let intBinTree =
  Node(43, Node(25, Node(56, Leaf, Leaf), Leaf),
        Node(562, Leaf, Node(78, Leaf, Leaf)))
```

we can do the following example:

```
> mapInOrder (fun x -> x+1) intBinTree;;
val it : int BinTree =
  Node
    (44, Node (26, Node (57, Leaf, Leaf), Leaf),
      Node (563, Leaf, Node (79, Leaf, Leaf)))
```

Can you give an example of why mapInOrder might give a result different from mapPostOrder, but the result tree returned in both cases is still the same.

Exercise 5.3 Write a function

```
foldInOrder : ('a -> 'b -> 'b) -> 'b -> 'a BinTree -> 'b
```

that makes an in-order traversal of the tree and folds over the elements.

For instance, given the tree

```
let floatBinTree = Node(43.0, Node(25.0, Node(56.0, Leaf, Leaf), Leaf),  
                        Node(562.0, Leaf, Node(78.0, Leaf, Leaf)))
```

the application

```
foldInOrder (fun n a -> a + n) 0.0 floatBinTree
```

returns 764.0.